# DATA STRUCTURES

## 🔴 Tier 1 — Must-Know (Very High Priority)

If you are weak here, clearing product-based interviews is extremely difficult.

### 1. Arrays

- Foundation of almost all problems
- Prefix sum, sliding window, two pointers
- Time/space trade-offs

### 2. Strings

- Parsing, hashing, pattern matching
- Palindromes, substrings, frequency maps

### 3. Hash Table (HashMap / HashSet)

- Constant-time lookup
- Frequency counting, caching, deduplication
- Backbone of optimized solutions

### 4. Linked List

- Pointer manipulation
- Reversal, cycle detection, merge
- Tests understanding of memory & references

### 5. Stack

- Expression evaluation
- Parentheses validation
- Monotonic stack problems (very common)

### 6. Queue / Deque

- BFS traversal
- Sliding window optimization
- Task scheduling problems

---

## 🟠 Tier 2 — Core Interview & System Thinking (High Priority)

Expected for **mid-level and senior roles**.

### 7. Binary Tree

- Traversals (DFS, BFS)
- Height, diameter, LCA
- Recursive thinking

### 8. Binary Search Tree (BST)

- Ordered data operations
- Range queries
- In-order traversal logic

### 9. Heap / Priority Queue

- Top-K problems
- Scheduling, load balancing
- Streaming data use cases

### 10. Graph (Adjacency List / Matrix)

- BFS, DFS
- Shortest path (unweighted)
- Connectivity problems

---

## 🟡 Tier 3 — Advanced but Frequently Asked (Medium Priority)

Strong signal of **good problem-solving depth**.

### 11. Trie (Prefix Tree)

- Autocomplete
- Dictionary search
- String optimization problems

### 12. Disjoint Set (Union–Find)

- Cycle detection
- Connected components
- Network problems

### 13. Segment Tree

- Range queries

- Competitive programming style problems
- Seen in high-bar interviews

### 14. Binary Indexed Tree (Fenwick Tree)

- Efficient cumulative frequency
- Less common but valuable

---

## 🟢 Tier 4 — Specialized / Role-Dependent (Low Priority)

Good to know, rarely a blocker.

### 15. Balanced Trees (AVL, Red-Black Tree)

- Mostly conceptual
- Libraries handle implementation in practice

### 16. Skip List

- Rare in interviews
- Sometimes discussed conceptually

### 17. Suffix Array / Suffix Tree

- Advanced string problems
- Mostly for research-heavy roles

### 18. B-Tree / B+ Tree

- Database internals
- Storage systems interviews

---

## 🔵 Tier 5 — Nice to Know (Very Low Priority)

Almost never directly tested.

### 19. Bloom Filter

- Probabilistic data structures
- Used in system design discussions

### 20. LRU Cache (Conceptual + Implementation)

- Combination of HashMap + Doubly Linked List
- Common as a **design problem**, not pure DS

---

## 📌 Interview Reality Check (Very Important)

Product companies do **NOT** test data structures in isolation.
They test:

- **When to use which DS**
- **Why one DS is better than another**
- **Time–space trade-offs**
- **Edge-case handling**

Example:

> "Use a HashMap + Heap" is more valuable than knowing Heap alone.

---

## 🎯 Practical Recommendation (Roadmap)

If you want a **high ROI preparation path**:

1. **Master Tier 1 completely**
2. Be **very strong in Tier 2**
3. Understand **when & why** Tier 3 is used
4. Know Tier 4 & 5 **conceptually**

---

# ALGORITHMS

## 🔴 Tier 1 — Must-Know Algorithms (Very High Priority)

If these are weak, clearing product interviews is unlikely.

### 1. Sorting Algorithms

- Merge Sort
- Quick Sort
- Counting Sort (conceptual)
- Why built-in sort works (Timsort / Hybrid)

- Stability, in-place vs out-of-place

### 2. Binary Search

- Standard binary search
- First/last occurrence
- Search on answer (binary search on range)
- Rotated arrays

### 3. Two Pointers Technique

- Sorted array problems
- Pair/triplet problems
- Fast/slow pointer patterns

### 4. Sliding Window

- Fixed window
- Variable window
- Frequency-based window problems

### 5. Hashing Techniques

- Frequency counting
- Prefix sums with hash maps
- Collision awareness (conceptual)

### 6. Recursion

- Base case clarity
- Stack behavior
- Convert recursion to iteration

---

## 🟠 Tier 2 — Core Problem-Solving Algorithms (High Priority)

Expected for **mid-level and senior engineers**.

### 7. Tree Traversal Algorithms

- DFS (pre/in/post order)
- BFS (level order)
- Iterative vs recursive

### 8. Graph Traversal Algorithms

- BFS
- DFS
- Cycle detection
- Connected components

### 9. Greedy Algorithms

- Interval scheduling
- Activity selection
- Optimal local decision reasoning

### 10. Heap-Based Algorithms

- Top-K problems
- K-way merge
- Scheduling with priority queues

---

## 🟡 Tier 3 — Advanced & High-Signal Algorithms (Medium Priority)

Strong differentiators in **hard interviews**.

### 11. Dynamic Programming (DP)

- 1D DP (climbing stairs, house robber)
- 2D DP (knapsack, grid problems)
- Optimization (space reduction)
- Bottom-up vs top-down

### 12. Backtracking

- Permutations & combinations
- Subsets
- Constraint-based exploration

### 13. Divide and Conquer

- Merge-based logic
- Inversion count
- Recursive partitioning

### 14. Union-Find (Disjoint Set Algorithms)

- Union by rank
- Path compression

- Cycle detection

---

## 🟢 Tier 4 — Specialized / Situational Algorithms (Low Priority)

Asked occasionally or role-dependent.

### 15. Shortest Path Algorithms

- Dijkstra (priority queue based)
- Bellman-Ford (conceptual)
- Floyd-Warshall (rare)

### 16. Topological Sort

- DAG scheduling
- Dependency resolution

### 17. String Matching Algorithms

- KMP (conceptual understanding)
- Rabin-Karp (hash-based matching)
- Z-algorithm (rare)

### 18. Bit Manipulation Algorithms

- XOR tricks
- Bit masking
- Power-of-two checks

---

## 🔵 Tier 5 — Nice-to-Know / Rare (Very Low Priority)

Almost never a deciding factor.

### 19. Mathematical Algorithms

- GCD / LCM
- Sieve of Eratosthenes
- Modular arithmetic basics

### 20. Advanced Graph Algorithms

- Max Flow / Min Cut
- Strongly Connected Components
- Bipartite matching

### 21. Computational Geometry

- Line sweep
- Convex hull

---

## 📌 Interview Reality (Important Insight)

Product companies **do not reward algorithm memorization**.
They evaluate:

- Pattern recognition
- Trade-off analysis
- Correctness + efficiency
- Edge-case handling
- Clean implementation

Example:

> Knowing *when* to use **DP vs Greedy** matters more than knowing DP formulas.

---

## 🎯 High-ROI Preparation Strategy

**Recommended order:**

1. Master **Tier 1**
2. Be strong in **Tier 2**
3. Learn **patterns** in Tier 3
4. Conceptual familiarity with Tier 4
5. Skim Tier 5